# Guida al sistema

Come è fatta la casa editrice del repository ZeusBot: chi fa cosa, il percorso di un libro, le regole.

Per Felice, autore. Ottobre 2026.

Fotografia del ramo `motore` al commit `f7decc4`.

> **Come leggere.** Tra parentesi, in grigio, c'è il file da cui viene ogni affermazione. Quando una cosa non c'è nel repository, la guida lo scrive: «non esiste».

# Indice

0. Le parole da sapere
1. A cosa serve il sistema
2. Organigramma: chi fa cosa
3. Il percorso di un libro
4. Le regole
5. I rami
6. Cosa c'è adesso
7. Appendici e indice
8. Cosa resta da fare all'autore
9. Dove trovare le cose

# 0. Le parole da sapere

Poche parole tecniche tornano spesso. Eccole, una volta per tutte.

- **Repository**: l'archivio del progetto su GitHub, con tutta la storia delle modifiche. Qui si chiama ZeusBot.
- **Ramo**: una versione parallela del repository. Ogni ramo ha i suoi file. Il ramo `motore` contiene la macchina; ogni libro vive su un ramo suo.
- **Commit**: un salvataggio con nome e data. Ha un codice breve, per esempio `f7decc4`.
- **Push**: l'invio dei salvataggi su GitHub (su «origin», la copia in rete). Finché non c'è il push, il lavoro esiste solo nel computer di lavoro.
- **Merge**: l'unione di un ramo dentro un altro.
- **Script**: un piccolo programma che fa un lavoro preciso (contare parole, fare il PDF…).
- **Hook**: un guardiano automatico. Claude Code lo chiama prima di certe

azioni; l'hook può lasciar passare, avvisare o bloccare.

- **Claude Code**: l'assistente che lavora nel repository, legge le regole e lancia i comandi.
- **YAML**: un formato di testo per i dati, fatto di righe «nome: valore». I file finiscono in `.yaml`.
- **Gate**: una fermata obbligata. Il lavoro si ferma finché l'autore non approva.
- **sha256**: l'«impronta digitale» di un file. Se cambia anche una virgola, l'impronta cambia.
- **KDP**: Kindle Direct Publishing, il servizio di Amazon per pubblicare.
- **Metadati del PDF**: dati nascosti nel file, come Title (titolo) e Author (autore).
- **Slug**: il nome breve di un libro usato nei percorsi, senza spazi né accenti.

# 1. A cosa serve il sistema

Il sistema è una fabbrica di libri. Serve per romanzi di qualsiasi genere (motore/README.md).

Il cuore è il **motore**: procedura, script, modelli vuoti, profili di genere e dati KDP. Il motore non contiene libri veri (motore/README.md).

Il percorso è sempre lo stesso. L'autore scrive un briefing. Il motore crea la cartella del libro. Claude Code scrive i documenti e poi i capitoli, uno alla volta. A ogni tappa importante l'autore approva. Alla fine il motore controlla il testo, compone il libro, fa il PDF di stampa e prepara il pacchetto per KDP (motore/README.md, «Dal briefing al PDF»).

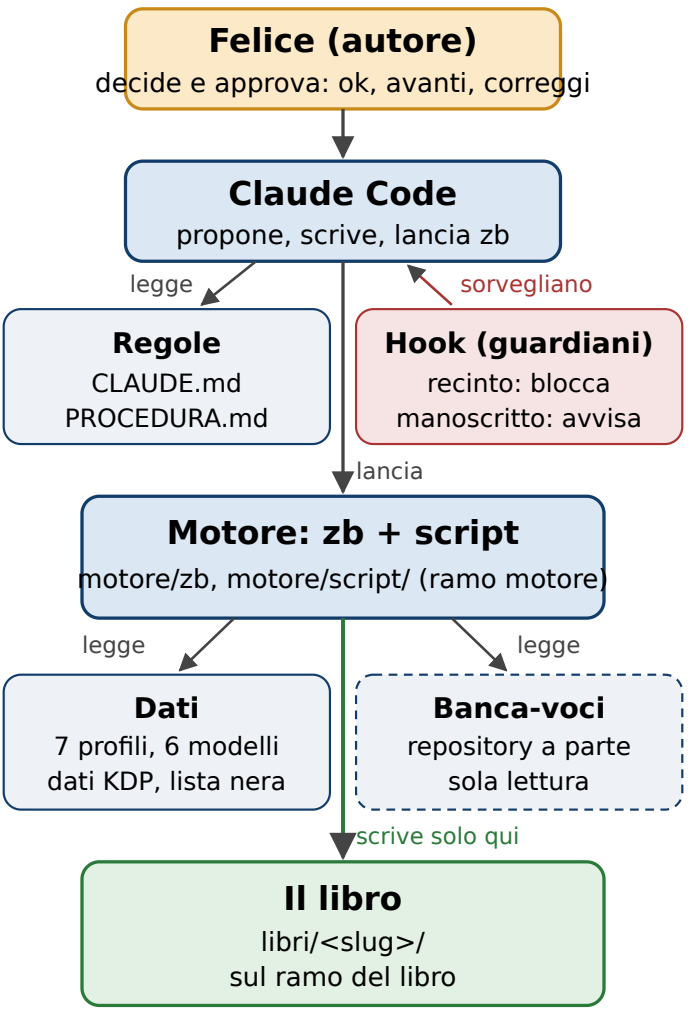
Tre idee reggono tutto (CLAUDE.md):

1. **Il motore resta pulito.** Il ramo `motore` non contiene mai un libro.

2. **Si scrive solo nella cartella del libro.** Mai nel motore, mai altrove.
3. **Proponi, l'autore approva, poi applica.** Claude Code non approva mai da solo.

# 2. Organigramma: chi fa cosa

## Lo schema



Schema ricavato da CLAUDE.md, motore/README.md e docs/INFRASTRUTTURA_COMPLETA.md §3 e §8.4.

## Chi fa cosa, in breve

| Chi | Che cosa fa | Dove sta |
|---|---|---|
| Felice | Scrive il briefing, risponde ok / avanti / correggi, decide | — |
| Claude Code | Legge le regole, propone, scrive nel libro, lancia `zb` | — |
| Motore | Crea, controlla, compone, impagina | `motore/` |

| Chi | Che cosa fa | Dove sta |
|---|---|---|
| Regole | Dicono come si lavora | `CLAUDE.md`, `motore/PROCEDURA.md` |
| Hook | Sorvegliano le scritture | `motore/script/hook_*.py` |
| Libro | Il lavoro vero | `libri/<slug>/` |

(CLAUDE.md; motore/README.md; docs/INFRASTRUTTURA_COMPLETA.md §8.4)

## Il motore: `zb` e gli script

`zb` è il comando unico. Si scrive `python3 motore/zb <comando> <libro>`. Il libro si indica sempre con il percorso della sua cartella (motore/zb; motore/README.md, «Comandi»).

Ogni comando di `zb` lancia uno script della cartella `motore/script/`. Gli script sono 28 file (motore/zb, tabella SCRIPT; cartella motore/script/).

I comandi principali (motore/README.md, «Comandi»):

- `zb nuovo`: crea il libro dal briefing.
- `zb avvio`: controlli di inizio sessione e riga «Letto: …».
- `zb stato`: mostra a che punto è il libro. Non scrive nulla.
- `zb pronto`: chiude il lavoro di una fase e apre la fermata.
- `zb ok`, `zb avanti`, `zb correggi`: le risposte dell'autore.
- `zb esito`: controlla un capitolo appena scritto.
- `zb continuita`, `zb ortografia`, `zb compila`, `zb impagina`, `zb pdf`, `zb pacchetto`, `zb kdp`: la chiusura.
- `zb prove`: le prove del motore sui mini-libri inventati.

Nessun comando del motore fa commit o push da solo (motore/PROCEDURA.md §8).

## La «Bibbia editor»

Nel repository «Bibbia editor» è il nome dato all'insieme delle regole editoriali valide per ogni libro. Il commit `668afc2` si chiama proprio «Bibbia editor: appendici e indice» e modifica `motore/PROCEDURA.md` (docs/INFRASTRUTTURA_COMPLETA.md §4).

Le regole stanno in due file (docs/INFRASTRUTTURA_COMPLETA.md §4):

- `CLAUDE.md`: le regole di avvio. Claude Code le legge a ogni sessione.
- `motore/PROCEDURA.md`: la procedura completa, in 11 sezioni.

Le aiutano alcuni file di dati, per esempio la lista nera di base (`motore/dati/lista-nera.yaml`) e le parole italiane ammesse (`motore/dati/parole-ammesse-it.txt`) (docs/INFRASTRUTTURA_COMPLETA.md §4).

> Attenzione: la `bibbia.md` di ogni libro (personaggi, luoghi, fatti) è un'altra cosa. Sta nella cartella del libro, in `02-bibbia/` (docs/INFRASTRUTTURA_COMPLETA.md §4).

## Profili di genere

Un profilo dà i valori predefiniti di un genere: lunghezza, capitoli, scene, frase media, dialogo (motore/PROCEDURA.md §3).

Sono 7, nella cartella `motore/profili/`: fantascienza, fantasy, giallo, narrativa letteraria, romance, thriller, young adult (cartella motore/profili/).

Un valore del briefing fuori dal profilo è accettato solo con un «Motivo override:» scritto nel briefing. Senza motivo, `zb nuovo` si ferma (motore/PROCEDURA.md §3).

## Voci narranti

Una voce dice **come** si racconta. Il **cosa** lo decide l'autore nel briefing (motore/README.md, «Banca-voci»).

Le voci stanno in un repository separato, la banca-voci, mai dentro il motore. Il motore le legge in sola lettura dalla cartella indicata da `ZB_BANCA_VOCI` (motore/README.md, «Banca-voci»).

Oggi le voci sono 11, tutte pronte. L'elenco è in `indice.yaml` della banca-voci e in `motore/passaggio-di-consegne.md`, «Stato» (motore/passaggio-di-consegne.md). Questa guida non le nomina.

Ogni voce ha una scheda in 7 sezioni e un elenco di espressioni vietate, `divieti.yaml` (motore/README.md, «Banca-voci»).

Un libro sceglie la voce nel briefing («Voce dalla banca:», «Ambientazione:»); `zb nuovo` la scrive in `libro.yaml` (motore/README.md, «Banca-voci»).

Se il briefing contrasta con la voce, vince il briefing, e il contrasto si segnala (motore/modelli/briefing.md; README della banca-voci, copiato in docs/INFRASTRUTTURA_COMPLETA.md §6).

## Modelli

I modelli sono i file vuoti da cui nasce un libro. Stanno in `motore/modelli/` e sono 6 (cartella motore/modelli/):

| File | A cosa serve |
|---|---|
| `briefing.md` | Il modulo che compila l'autore |
| `libro.yaml` | Le regole del libro |
| `stato.yaml` | A che punto è il libro |
| `cronologia.yaml` | Date ed eventi |
| `LEGGIMI.md` | Cosa leggere a ogni avvio |
| `conferme-autore.yaml` | Le conferme per KDP |

(motore/modelli/; docs/INFRASTRUTTURA_COMPLETA.md §3.3)

## Hook

Gli hook sono tre. Stanno nel modello `motore/dati/hook-settings.esempio.json` (motore/dati/hook-settings.esempio.json):

| Hook | Quando | Cosa fa |
|---|---|---|
| recinto | prima di ogni scrittura di file | **Blocca** le scritture fuori dal libro attivo |
| manoscritto | prima di scrivere in `04-manoscritto/` | Oggi **avvisa** se manca `zb avvio` |
| sessione | all'inizio della sessione | Annota la sessione |

(motore/script/hook_recinto.py; motore/script/hook_manoscritto.py; motore/dati/hook.yaml; motore/PROCEDURA.md §10)

Gli hook si accendono solo sui rami dei libri, con `zb hook attiva --ok`. Sul ramo `motore` sono spenti per scelta (motore/passaggio-di-consegne.md, «Stato»). Dettagli nel capitolo 4.

## CLAUDE.md

È il foglio delle regole di avvio. Sta nella radice del repository e Claude Code lo legge a ogni sessione (docs/INFRASTRUTTURA_COMPLETA.md §4 e §9).

Ha al massimo 30 righe e nessun nome di libro: lo controlla una prova del motore (docs/INFRASTRUTTURA_COMPLETA.md §9; motore/script/prove.py). Cosa dice: capitolo 4.

## Rami dei libri

Ogni libro vive su un ramo suo, nella cartella `libri/<slug>/`. Quel ramo porta anche una copia del motore (docs/INFRASTRUTTURA_COMPLETA.md §8.3). Dettagli nel capitolo 5.

# 3. Il percorso di un libro

Il libro attraversa dodici tappe; dove c'è un gate il lavoro si ferma finché l'autore non scrive «ok» (motore/editoriale/00-percorso.md).

| # | Tappa | Chi lavora | Fermata |
|---|---|---|---|
| 1 | Briefing | autore | — |
| 2 | Cartella unica `libri/<slug>/` | `zb nuovo` | — |
| 3 | Struttura: nove file | architetto della struttura | — |
| 4 | Revisione della struttura | diagnosta, poi editor strutturale | `struttura` |

| # | Tappa | Chi lavora | Fermata |
|---|---|---|---|
| 5 | Documenti: bibbia, codex, cronologia | direttore editoriale | `documenti` |
| 6 | Pagina campione | scrittore con voce | `pagina_campione` |
| 7 | Stesura, un capitolo alla volta | scrittore, poi capsula | `primi_capitoli` |
| 8 | Revisione strutturale del manoscritto | diagnosta, poi editor strutturale | `struttura_manoscritto` |
| 9 | Line editing | line editor | `line` |

| # | Tappa | Chi lavora | Fermata |
|---|---|---|---|
| 10 | Copyediting | correttore di italiano | `copy` |
| 11 | Bozze | correttore di bozze | `bozze` |
| 12 | Chiusura meccanica | motore | — |

## Le risposte dell'autore

`avanti` mostra il blocco successivo; `ok` approva; `correggi: …` chiede una correzione; `stato` mostra dove si è. Nelle tappe 8-11 l'«ok» approva le proposte: solo dopo si applicano, con modifiche puntuali.

## Prima di ogni sessione: `zb avvio`

Stampa la riga «Letto» e si ferma se qualcosa non va: modifiche non salvate, file fuori dal libro, un altro libro attivo,

documenti approvati cambiati. In una sessione nuova si legge lo stato compatto del libro (15-20 righe) e la scheda del capitolo.

## La stesura

`/scrivi-capitolo N` prepara il capitolo (scheda, schede dei personaggi e dei luoghi in scena, fatti bloccati, capsule), chiama lo scrittore, controlla il capitolo, scrive la capsula e salva. Poi si ferma: la revisione è un altro comando (`/revisiona`), in un'altra fase.

## La chiusura

`zb continuita`, `zb ortografia`, `zb compila`, `zb impagina`, `zb pdf`, `zb pacchetto`, `zb kdp`, poi `zb pronto`.

## La traduzione

`zb traduci <libro> --a en inizia`: guida e glossario si approvano prima del capitolo 1; ogni capitolo tradotto ha i suoi controlli (paragrafi, battute, numeri,

nomi, parole rimaste in italiano, lunghezza, nessuna nota aggiunta); poi line, copy e revisione di fedeltà. KDP chiede di dichiarare una traduzione fatta con l'IA (pagina ufficiale, verificata il 2026-10-15). Per un'edizione in vendita è consigliata la lettura finale di un madrelingua.

## Dopo ogni passo: il salvataggio

`zb salva`: controlla che nessun file stia fuori dal libro, salva e invia su GitHub.

# 4. Le regole

## Cosa dice CLAUDE.md

Una cartella per libro; prima `zb avvio`; la sessione principale non scrive prosa; chi scrive non rivede; report di sessione ogni 5 scambi o a «passaggio»; una sola scala di precedenza; salvataggio con `zb salva` (CLAUDE.md).

## Il recinto

Blocca le scritture fuori dal libro attivo: con l'hook (anche sui comandi Bash), con il controllo che git fa prima di ogni salvataggio, con `zb avvio` e `zb salva`. Ciò che l'hook non vede (uno script che scrive senza nominare il file) lo ferma il controllo di git. Sul ramo del motore hook, agenti e comandi restano spenti.

## L'ordine di precedenza

1. manuale del libro e `libro.yaml`; 2. briefing e scheda della voce (se contrastano vince il briefing, e si segnala); 3. profilo di genere; 4.

`motore/editoriale/`; 5. procedura del motore. Il catalogo anti-IA e i criteri del correttore sono filtri: non stanno sopra il manuale (motore/editoriale/LEGGIMI.md).

# 5. I rami

## Il ramo `motore`

Solo il motore: codice, regole, mini-libri inventati. Nessun libro, nessun nome di libro o di autore reale (lo controlla `zb separazione`, anche in `docs/` e in `CLAUDE.md`).

## I rami dei libri

Un libro per ramo, in `libri/<slug>/`. I libri vecchi hanno il loro CLAUDE.md per scelta dell'autore e non si toccano.

# 6. Cosa c'è adesso

- **Subagenti e comandi** (`/scrivi-capitolo`, `/revisiona`, `/diagnosi`, `/handover`): si accendono sul ramo del libro con `zb agenti attiva --ok`.
- **Anti-IA**: controllo che misura e non riscrive (`zb esito`, `zb antiia`). Per l'italiano nessuna lista di parole finché non si estrae dai dati.
- **Correttore di italiano**: criteri, passaggio per capitolo, Hunspell e LanguageTool (`zb languagetool installa`).
- **Inglese**: un libro si può scrivere direttamente in inglese con gli stessi ruoli e gate.

# 7. Appendici e indice

Le appendici (soluzioni, mappe, glossari) stanno in `05-appendici/` e si dichiarano in `libro.yaml`; il sommario lo genera `zb compila` (motore/PROCEDURA.md, sezione 7).

# 8. Cosa resta da fare all'autore

- Verificare i dati KDP e registrare la verifica (`zb verifica KDP fatta AAAA-MM-GG`).
- Fornire, se si vuole, un corpus per il vocabolario anti-IA italiano.
- Decidere sulla licenza non commerciale di un repository esterno (APODICTIC).
- Elenco aggiornato: `docs/DA_FORNIRE.md`.

# 9. Dove trovare le cose

Ramo «motore» vuol dire anche: nella copia del motore di ogni ramo di libro aggiornato.

| Cosa cerchi | File | Ramo |
|---|---|---|
| Regole di avvio | `CLAUDE.md` | motore (ogni ramo di libro ha il suo) |
| Procedura completa | `motore/PROCEDURA.md` | motore |
| Elenco dei comandi | `motore/README.md`, «Comandi»; `python3 motore/zb aiuto` | motore |
| Stato del lavoro sul motore | `motore/passaggio-di-consegne.md` | motore |
| Limiti noti del motore | `motore/report-motore.md` §11 | motore |

| Cosa cerchi | File | Ramo |
|---|---|---|
| Progetto originale | `motore/proposta-motore.md` | motore |
| Profili di genere | `motore/profili/` | motore |
| Modelli vuoti | `motore/modelli/` | motore |
| Modulo del briefing | `motore/modelli/briefing.md` | motore |
| Dati KDP | `motore/dati/kdp.yaml` | motore |
| Lista nera | `motore/dati/lista-nera.yaml` | motore |
| Parole italiane ammesse | `motore/dati/parole-ammesse-it.txt` | motore |
| Cartelle di un libro | `motore/dati/struttura-libro.yaml` | motore |
| Campi di `libro.yaml` | `motore/dati/libro.schema.yaml` | motore |

| Cosa cerchi | File | Ramo |
|---|---|---|
| Hook | `motore/script/hook_*.py`, `motore/dati/hook-settings.esempio.json` | motore |
| Modalità dell'hook del manoscritto | `motore/dati/hook.yaml` | motore |
| Hook accesi | `.claude/settings.json` | ramo del libro |
| Impaginazione e font | `motore/stampa/` | motore |
| Prove e mini-libri inventati | `motore/script/prove.py`, `motore/prove/` | motore |
| Libro attivo | `libri/libro-attivo.md` | ramo del libro |
| Un libro | `libri/<slug>/` | ramo del libro |

| Cosa cerchi | File | Ramo |
|---|---|---|
| Briefing del libro | `libri/<slug>/00-progetto/briefing.md` | ramo del libro |
| Bibbia del libro | `libri/<slug>/02-bibbia/bibbia.md` | ramo del libro |
| Capitoli | `libri/<slug>/04-manoscritto/` | ramo del libro |
| Appendici | `libri/<slug>/05-appendici/` | ramo del libro |
| PDF e libro compilato | `libri/<slug>/05-output/` | ramo del libro |
| Report dei controlli | `libri/<slug>/06-diagnostica/` | ramo del libro |
| Pacchetto KDP | `libri/<slug>/06-pubblicazione/` | ramo del libro |
| Report di sessione del libro | `libri/<slug>/passaggio-di-consegne.md` | ramo del libro |

| Cosa cerchi | File | Ramo |
|---|---|---|
| Voci narranti | `indice.yaml`, `voci/<slug>/` | banca-voci, ramo `main` (repository a parte) |
| Archivio completo | `docs/INFRASTRUTTURA_COMPLETA.md` | motore |
| Questa guida | `docs/GUIDA_AL_SISTEMA.pdf` e `.md` | motore |

(docs/INFRASTRUTTURA_COMPLETA.md §8.4 e §9; motore/dati/struttura-libro.yaml; motore/README.md)

Fine della guida.